Part 1

Q1: The query returns 0 because NULL = NULL evaluates to unknown; the correct query is SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;. NULL represents unknown data, making aggregate calculations logically impossible.

Q2: Yes, the student is correct because GROUP BY department already outputs exactly one row per department, making DISTINCT redundant.

Q3: The row count difference indicates that there are customers who have not placed any orders. A LEFT JOIN returns more rows when the left table (Customer) contains records with no matches in the right table (Order).

Q4: The query is invalid because employee_name is neither in the GROUP BY clause nor within an aggregate function; the correct query is SELECT department, AVG(salary) FROM Employee GROUP BY department;. If allowed, SQL would not know which specific employee's name to arbitrarily display for the aggregated department row.

Q5: The error is using WHERE to evaluate an aggregate condition instead of HAVING. WHERE filters rows before aggregation occurs, so the corrected query is SELECT department, AVG(salary) FROM Employee GROUP BY department HAVING AVG(salary) > 80000;.

Q6 (a): Non-correlated, because the overall average is a static value that only needs to be calculated once.

Q6 (b): Correlated, because the average must be dynamically recalculated for each employee's specific department.

Q6 Key Implication: Correlated subqueries heavily degrade performance on large tables because they execute once for every row in the outer table.

Part 2

- **(a)** SELECT c.* FROM Customer c LEFT JOIN Order o ON c.customer_id = o.customer_id WHERE o.order_id IS NULL;. A LEFT JOIN combined with an IS NULL filter correctly isolates records in the left table that have no matches.
- **(b)** **(b)** SELECT o.restaurant_id, AVG(oi.quantity * m.price) FROM Order o JOIN OrderItem oi ON o.order_id = oi.order_id JOIN MenuItem m ON oi.item_id = m.item_id GROUP BY o.restaurant_id HAVING COUNT(DISTINCT o.order_id) > 5;. HAVING is required to filter the grouped restaurants by their order count after aggregation.
- **(c)** **(c)** WITH R_Avg AS (SELECT o.restaurant_id, AVG(oi.quantity * m.price) as v FROM Order o JOIN OrderItem oi ON o.order_id = oi.order_id JOIN MenuItem m ON oi.item_id = m.item_id GROUP BY o.restaurant_id) SELECT restaurant_id FROM R_Avg WHERE v > (SELECT AVG(v) FROM R_Avg);. A non-correlated subquery is used because the overall platform average is calculated just once.
- **(d)** SELECT restaurant_id, item_id FROM (SELECT o.restaurant_id, oi.item_id, SUM(oi.quantity), ROW_NUMBER() OVER(PARTITION BY o.restaurant_id ORDER BY SUM(oi.quantity) DESC) as rn FROM Order o JOIN OrderItem oi ON o.order_id = oi.order_id GROUP BY o.restaurant_id, oi.item_id) t WHERE rn = 1;. A core limitation of basic GROUP BY + MAX() is that it loses the association with other columns that produced the maximum value.

Part 3

Q1: The dependencies are student_id -> student_name, course_id -> course_name, instructor_id, instructor_id -> instructor_name, instructor_office, and (student_id, course_id) -> grade, enrollment_date

Q2: Yes, assuming all stored attributes contain indivisible, atomic values.

Q3: student_id -> student_name is a partial dependency. Changing a student's name creates an update anomaly because every course enrollment record for that student must be redundantly updated.

Q4: course_id -> instructor_id -> instructor_office is a transitive dependency. Deleting a course causes a deletion anomaly if it inadvertently removes the only existing record of an instructor's office.

Q5:

- Student(**student_id**, student_name) - Separated to resolve the partial dependency on student_id.
- Instructor(**instructor_id**, instructor_name, instructor_office) - Separated to resolve the transitive dependency.
- Course(**course_id**, course_name, instructor_id) - Separated to resolve the partial dependency on course_id.
- Enrollment(**student_id, course_id**, grade, enrollment_date) - The base relation for the composite key

Part 4

In Part II (a), a NOT IN subquery could have been used, but LEFT JOIN was chosen to avoid potential logical errors and missing records when evaluating NULL values inside subqueries.

Normalizing into 3NF makes querying harder by requiring complex JOIN operations to reconstruct a full enrollment record, meaning real systems may retain some denormalization to optimize read speeds for frequently accessed data.